EXPERIMENT 1: TITANIC DATASET PREPROCESSING AND VISUALIZATION

AIM:

To perform data preprocessing and exploratory data analysis on the Titanic dataset by inspecting data, handling missing values and duplicates, encoding categorical data, scaling numerical data, and visualizing feature relationships.

ALGORITHM:

1. Import the required Python libraries: Pandas, Seaborn, Matplotlib, and scikit-learn preprocessing tools.
2. Load the Titanic CSV dataset into a Pandas DataFrame and inspect the first rows, data types, and missing values.
3. Fill missing Cabin values with “Unknown” and remove duplicate records.
4. Encode the Sex column using LabelEncoder.
5. Standardize the Fare column using StandardScaler.
6. Fill missing Age values using forward fill and backward fill.
7. Plot selected features (Pclass, Sex, Age, SibSp) using a pair plot.
8. Calculate and display the correlation heatmap for Pclass, Age, SibSp, Parch, and Fare.

1. Load the Titanic dataset and convert it into a DataFrame.

```
#EXP_NO_1
#MUTHURAJ_241801176

import numpy as np
import pandas as pd

df = pd.read_csv('titanic.csv')
data = pd.DataFrame(df)
df.head()
```

	Survived	Pclass	Name	Sex	Age	Siblings/Spouses Aboard	Parents/Children Aboard	Fare
0	0	3	Mr. Owen Harris Braund	male	22.0	1	0	7.2500
1	1	1	Mrs. John Bradley (Florence Briggs Thayer) Cum...	female	38.0	1	0	71.2833
2	1	3	Miss. Laina Heikkinen	female	26.0	0	0	7.9250
3	1	1	Mrs. Jacques Heath (Lily May Peel) Futrelle	female	35.0	1	0	53.1000
4	0	3	Mr. William Henry Allen	male	35.0	0	0	8.0500

Next steps: Generate code with df | New interactive sheet

2. Explore and Understand the Dataset

- Display the first few rows.
- Get information about column data types and missing values.

```
#EXP_NO_1
#MUTHURAJ_241801176

df.head()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 887 entries, 0 to 886
Data columns (total 8 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Survived                887 non-null    int64
 1   Pclass                  887 non-null    int64
 2   Name                    887 non-null    object
 3   Sex                     887 non-null    object
 4   Age                     887 non-null    float64
 5   Siblings/Spouses Aboard 887 non-null    int64
 6   Parents/Children Aboard 887 non-null    int64
 7   Fare                    887 non-null    float64
dtypes: float64(2), int64(4), object(2)
memory usage: 55.6+ KB
```

3. Fill missing values in Age using Forward Fill and Backward Fill

```
#EXP_NO_1
#MUTHURAJ_241801176

df["Age"] = df["Age"].ffill()
df["Age"] = df["Age"].bfill()
df.head(10)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	35.0	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

4. Fill missing values in Cabin with "Unknown" (limit = 5)

```
#EXP_NO_1
#MUTHURAJ_241801176
df["Cabin"] = df["Cabin"].fillna("Unknown", limit = 5)
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	Unknown	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	Unknown	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	Unknown	S

Next steps: Generate code with df | New interactive sheet

5. Remove Duplicate Records if any

```
#EXP_NO_1
#MUTHURAJ_241801176
df.drop_duplicates(inplace=True)
df.head(10)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	Unknown	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	Unknown	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	Unknown	S
5	6	0	3	Moran, Mr. James	male	35.0	0	0	330877	8.4583	Unknown	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	Unknown	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	Unknown	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	Unknown	C

Next steps: Generate code with df | New interactive sheet

6. Encode the "Sex" column using LabelEncoder

```
#EXP_NO_1
#MUTHURAJ_241801176
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	1	22.0	1	0	A/5 21171	7.2500	Unknown	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	0	26.0	0	0	STON/O2. 3101282	7.9250	Unknown	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	1	35.0	0	0	373450	8.0500	Unknown	S

Next steps: Generate code with df | New interactive sheet

7. Scale the Fare column using StandardScaler

```
#EXP_NO_1
#MUTHURAJ_241801176
from sklearn.preprocessing import StandardScaler
s = StandardScaler()
df["Fare"] = s.fit_transform(df[["Fare"]])
df.head()
```

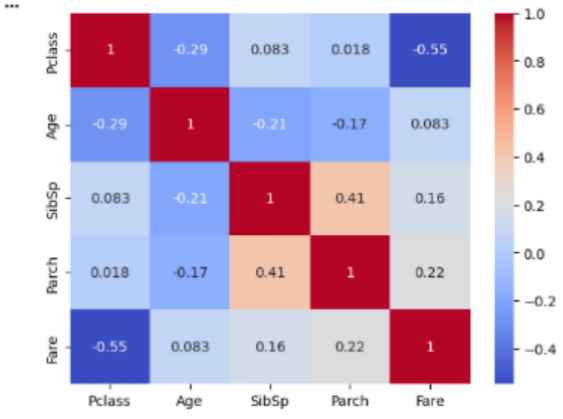
	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	1	22.0	1	0	A/5 21171	-0.502445	Unknown	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	38.0	1	0	PC 17599	0.786845	C85	C
2	3	1	3	Heikkinen, Miss. Laina	0	26.0	0	0	STON/O2. 3101282	-0.488854	Unknown	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	35.0	1	0	113803	0.420730	C123	S
4	5	0	3	Allen, Mr. William Henry	1	35.0	0	0	373450	-0.486337	Unknown	S

Next steps: Generate code with df | New interactive sheet

9. Display the Correlation Heatmap for 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare'

```
#EXP_NO_1
#MUTHURAJ_241801176

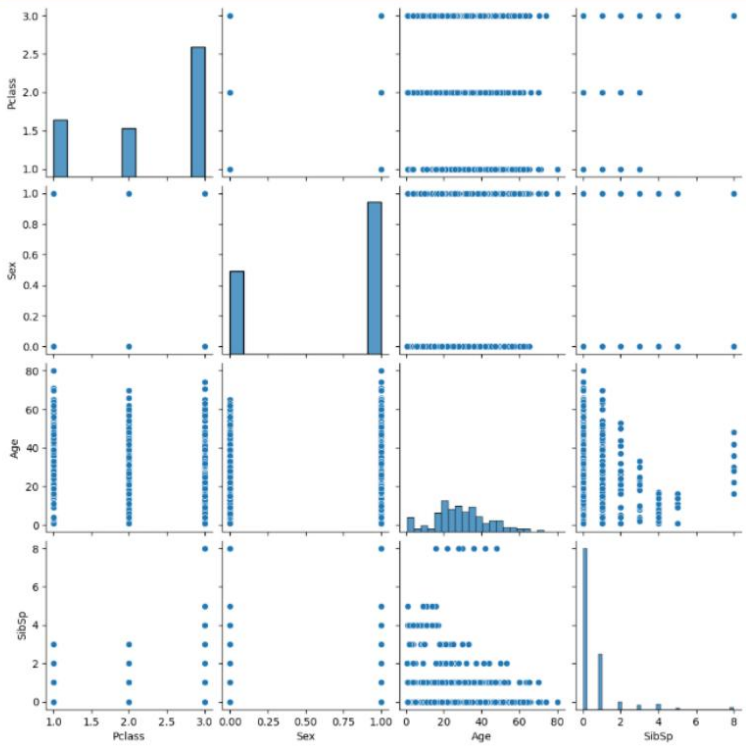
corr = df[["Pclass", "Age", "SibSp", "Parch", "Fare"]].corr()
sns.heatmap(corr, annot = True, cmap = "coolwarm")
plt.show()
```



8. Pair Plot of Selected Features such as 'Pclass', 'Sex', 'Age', 'SibSp'

```
#EXP_NO_1
#MUTHURAJ_241801176
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(df[["Pclass", "Sex", "Age", "SibSp"]])
plt.show()
```



RESULT:

The Titanic dataset was loaded into a DataFrame and explored. Missing values were handled, duplicate records were removed, the Sex column was label-encoded, the Fare column was standardized, and visualizations (pair plot and correlation heatmap) were generated to examine feature distributions and relationships.

EXPERIMENT 2: TITANIC DATASET ANALYSIS

AIM:

To analyze the Titanic dataset using Python by inspecting dataset information and summary statistics, handling missing values, visualizing passenger-class distribution, filtering passengers based on specified conditions, and splitting the data into training and testing sets.

ALGORITHM:

1. Import the required libraries: Pandas, Seaborn, Matplotlib, SimpleImputer, and train_test_split.

2. Load the Titanic CSV dataset into a Pandas DataFrame and display the first five rows.

3. Display the dataset shape, information, and summary statistics.

4. Fill missing values in the Age column using SimpleImputer with the mean strategy.

5. Replace missing values in the Cabin column with “Unknown”.

6. Fill missing values in the Embarked column using its mode.

7. Plot the passenger class distribution using a count plot.

8. Display female passengers who survived.

9. Display third-class passengers younger than 18 years.

10. Display first-class passengers older than 40 years.

11. Display surviving passengers in the specified category.

12. Display male passengers whose fare is greater than 100.

13. Display passengers who embarked from C and belonged to passenger class 2.

14. Display passengers with SibSp greater than 2.

15. Display non-surviving passengers who had no siblings/spouses and no parents/children aboard.

16. Display the five oldest surviving passengers.

17. Display passengers who paid zero fare.

18. Split the dataset into training and testing sets with test_size=0.2 and random_state=42.

```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 1. Import Required Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 2. Load the Telco Customer Churn Dataset
df = pd.read_csv("C:\\Users\\ADMIN\\Downloads\\archive (1)\\Telco-Customer-Churn.csv")
print("Dataset Loaded Successfully!")
```

```
Dataset Loaded Successfully!
```

```
print("\nFirst 10 Records")
print(df.head(10))
```

```
First 10 Records
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes
5  9305-CDSKC  Female              0      No         No       8          Yes
6  1452-KIOVK    Male              0      No        Yes      22          Yes
7  6713-OKOMC  Female              0      No         No      10           No
8  7892-POOKP  Female              0     Yes         No      28          Yes
9  6388-TABGU    Male              0      No        Yes      62          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No
5               Yes     Fiber optic             No  ...              Yes
6               Yes     Fiber optic             No  ...               No
```

```
              PaymentMethod MonthlyCharges  TotalCharges Churn
0          Electronic check          29.85         29.85    No
1              Mailed check          56.95        1889.5    No
2              Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4          Electronic check          70.70        151.65   Yes
5          Electronic check          99.65         820.5   Yes
6    Credit card (automatic)          89.10        1949.4    No
7              Mailed check          29.75         301.9    No
8          Electronic check         104.80       3046.05   Yes
9  Bank transfer (automatic)          56.15       3487.95    No

[10 rows x 21 columns]
```

```
print("\nLast 10 Records")
print(df.tail(10))
```

```
Last 10 Records
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
7033  9767-FFLEM    Male              0      No         No      38
7034  0639-TSIQW  Female              0      No         No      67
7035  8456-QDAVC    Male              0      No         No      19
7036  7750-EYXWZ  Female              0      No         No      12
7037  2569-WGERO  Female              0      No         No      72
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService       OnlineSecurity  ...  \
7033          Yes                No     Fiber optic                   No  ...
7034          Yes               Yes     Fiber optic                  Yes  ...
7035          Yes                No     Fiber optic                   No  ...
7036           No  No phone service             DSL                   No  ...
7037          Yes                No              No  No internet service  ...
7038          Yes               Yes             DSL                  Yes  ...
7039          Yes               Yes     Fiber optic                   No  ...
7040           No  No phone service             DSL                  Yes  ...
7041          Yes               Yes     Fiber optic                   No  ...
```

```
print("\nColumn Names")
print(df.columns)
```

```
Column Names
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='str')
```

```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 3. Explore the Dataset
print("\nDataset Information")
print(df.info())
```

```
Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   str
 1   gender            7043 non-null   str
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   str
 4   Dependents        7043 non-null   str
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   str
 7   MultipleLines     7043 non-null   str
 8   InternetService   7043 non-null   str
 9   OnlineSecurity    7043 non-null   str
 10  OnlineBackup      7043 non-null   str
 11  DeviceProtection  7043 non-null   str
 12  TechSupport       7043 non-null   str
 13  StreamingTV       7043 non-null   str
 14  StreamingMovies   7043 non-null   str
 15  Contract          7043 non-null   str
 16  PaperlessBilling  7043 non-null   str
```

```
print("\nSummary Statistics")
print(df.describe(include='all'))
```

```
Summary Statistics
       customerID gender  SeniorCitizen Partner Dependents       tenure  \
count        7043   7043    7043.000000    7043       7043  7043.000000
unique       7043      2            NaN       2          2          NaN
top     7590-VHVEG   Male            NaN      No         No          NaN
freq            1   3555            NaN    3641       4933          NaN
mean          NaN    NaN       0.162147     NaN        NaN    32.371149
std           NaN    NaN       0.368612     NaN        NaN    24.559481
min           NaN    NaN       0.000000     NaN        NaN     0.000000
25%           NaN    NaN       0.000000     NaN        NaN     9.000000
50%           NaN    NaN       0.000000     NaN        NaN    29.000000
75%           NaN    NaN       0.000000     NaN        NaN    55.000000
max           NaN    NaN       1.000000     NaN        NaN    72.000000

       PhoneService MultipleLines InternetService OnlineSecurity  ...  \
count          7043          7043            7043           7043  ...
unique            2             3               3              3  ...
top             Yes            No     Fiber optic             No  ...
freq           6361          3390            3096           3498  ...
mean            NaN           NaN             NaN            NaN  ...
std             NaN           NaN             NaN            NaN  ...
min             NaN           NaN             NaN            NaN  ...
25%             NaN           NaN             NaN            NaN  ...
50%             NaN           NaN             NaN            NaN  ...
75%             NaN           NaN             NaN            NaN  ...
max             NaN           NaN             NaN            NaN  ...

       DeviceProtection TechSupport StreamingTV StreamingMovies  \
count              7043        7043        7043            7043
unique                3           3           3               3
top                  No          No          No              No
freq               3095        3473        2810            2785
mean                NaN         NaN         NaN             NaN
std                 NaN         NaN         NaN             NaN
min                 NaN         NaN         NaN             NaN
25%                 NaN         NaN         NaN             NaN
```

```
print("\nMissing Values")
print(df.isnull().sum())
```

```
Missing Values
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 4(c). Verify Missing Values
print(df.isnull().sum())
```

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
print("\nDuplicate Records")
print(df.duplicated().sum())
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

```
Duplicate Records
0
```

```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 4(a). Remove Duplicate Records
print("\nShape Before Removing Duplicates:", df.shape)
df.drop_duplicates(inplace=True)
print("Shape After Removing Duplicates:", df.shape)
```

```
Shape Before Removing Duplicates: (7043, 21)
Shape After Removing Duplicates: (7043, 21)
```

```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 4(b). Replace Missing Values
df.fillna(df.median(numeric_only=True), inplace=True)

for col in df.select_dtypes(include=['object', 'str']).columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```python
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 5. Encode Categorical Columns
le = LabelEncoder()
categorical_columns = [
    'gender',
    'Partner',
    'Dependents',
    'Contract',
    'PaymentMethod',
    'InternetService',
    'Churn'
]
for col in categorical_columns:
    df[col] = le.fit_transform(df[col])

print("\nEncoded Columns")
print(df[categorical_columns].head())
```

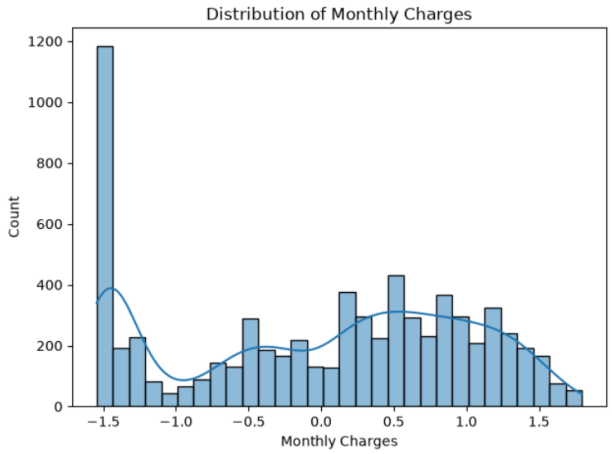
```
Encoded Columns
   gender  Partner  Dependents  Contract  PaymentMethod  InternetService  \
0       0        1           0         0              2                0
1       1        0           0         1              3                0
2       1        0           0         0              3                0
3       1        0           0         1              0                0
4       0        0           0         0              2                1

   Churn
0      0
1      0
2      1
3      0
```

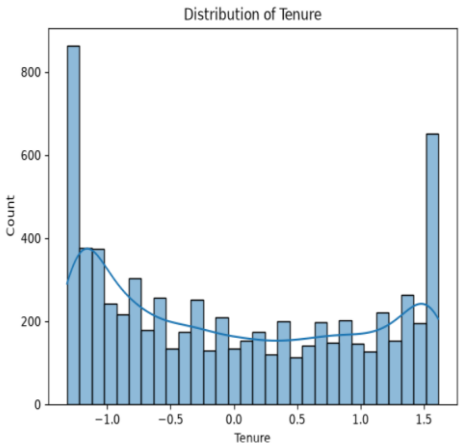
```python
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 6. Standardize Numerical Features
scaler = StandardScaler()
numerical_columns = [
    'tenure',
    'MonthlyCharges',
    'TotalCharges'
]
df[numerical_columns] = scaler.fit_transform(df[numerical_columns])
print("\nScaled Numerical Features")
print(df[numerical_columns].head())
```

```
Scaled Numerical Features
     tenure  MonthlyCharges  TotalCharges
0 -1.277445       -1.160323     -0.994242
1  0.066327       -0.259629     -0.173244
2 -1.236724       -0.362660     -0.959674
3  0.514251       -0.746535     -0.194766
4 -1.236724        0.197365     -0.940470
```

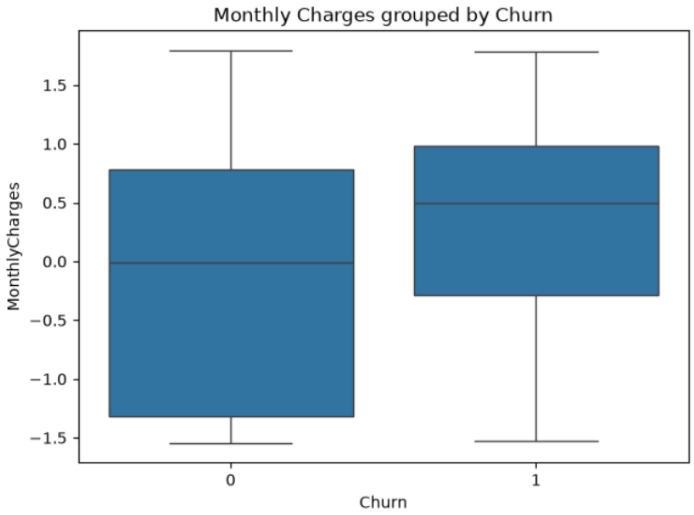
```python
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 7(a). Distribution of MonthlyCharges
plt.figure(figsize=(7,5))
sns.histplot(df['MonthlyCharges'], bins=30, kde=True)
plt.title("Distribution of Monthly Charges")
plt.xlabel("Monthly Charges")
plt.ylabel("Count")
plt.show()
```



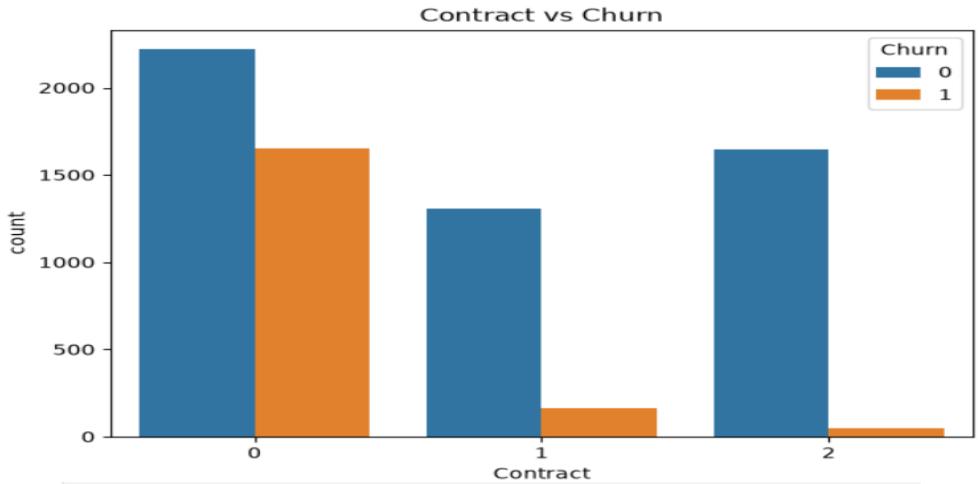

```python
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 7(b). Distribution of Tenure
plt.figure(figsize=(7,5))
sns.histplot(df['tenure'], bins=30, kde=True)
plt.title("Distribution of Tenure")
plt.xlabel("Tenure")
plt.ylabel("Count")
plt.show()
```



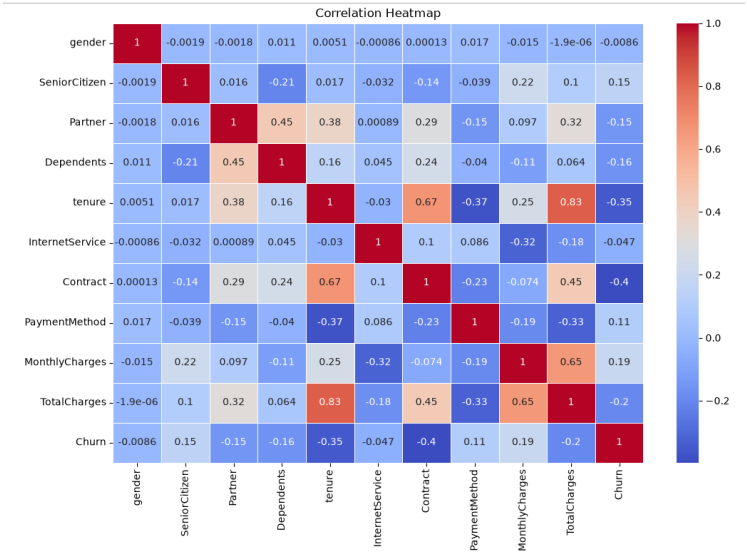
```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 9. Box Plot
plt.figure(figsize=(7,5))
sns.boxplot(x='Churn', y='MonthlyCharges', data=df)
plt.title("Monthly Charges grouped by Churn")
plt.show()
```




```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 8. Count Plot
plt.figure(figsize=(7,5))
sns.countplot(x='Contract', hue='Churn', data=df)
plt.title("Contract vs Churn")
plt.show()
```




```
# PRACTISE EXPERIMENT-1 Muthuraj(241801176)
# 10. Correlation Heatmap
corr = df.select_dtypes(include=['int64', 'float64']).corr()
plt.figure(figsize=(12,8))
sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)
plt.title("Correlation Heatmap")
plt.show()
```



RESULT:

The Titanic dataset was successfully loaded and analyzed. Dataset information and summary statistics were displayed, missing values in Age, Cabin, and Embarked were handled, and passenger-class distribution was visualized. Passenger records were filtered using the specified conditions. Finally, the dataset was split into 712 training records and 179 testing records.

EXPERIMENT 3: INFORMATION RETRIEVAL USING spaCy

AIM:

To implement an information retrieval system using spaCy and TF-IDF by preprocessing text, removing stop words, analyzing part-of-speech tags, and retrieving the most relevant documents for a user query using cosine similarity.

ALGORITHM:

1. Import spaCy, Pandas, fetch_20newsgroups, TfidfVectorizer, and cosine_similarity.
2. Load the 20 Newsgroups dataset and select the first 500 documents, excluding headers, footers, and quoted text.
3. Define a preprocessing function to convert text to lowercase, tokenize it with spaCy, remove stop words and non-alphabetic tokens, and use lemmas.
4. Store the original text and preprocessed text in a Pandas DataFrame.
5. Extract stop words from the documents using spaCy and display sample results.
6. Print part-of-speech tags for tokens in the first three documents.
7. Convert the cleaned documents into TF-IDF vectors using TfidfVectorizer.
8. Enter the query “NASA space launch” and preprocess it using the same text-preprocessing function.
9. Transform the query into a TF-IDF vector and calculate cosine similarity between the query and all documents.
10. Sort the similarity scores and display the top five most relevant documents.

```python
#MUTHURAJ_R_241801176
# EXP 3 Information Retrieval using spaCy

import spacy
import pandas as pd
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nlp = spacy.load("en_core_web_sm")
newsgroups = fetch_20newsgroups(
    subset="all",
    remove=("headers", "footers", "quotes")
)
documents = newsgroups.data[:500]

def preprocess(text):
    doc = nlp(text.lower())
    words = []
    for token in doc:
        if token.is_alpha and not token.is_stop:
            words.append(token.lemma_)
    return " ".join(words)

df = pd.DataFrame({"Text": documents})
df["clean_text"] = df["Text"].apply(preprocess)

def get_stop_words(text):
    doc = nlp(text.lower())
    words = []
    for token in doc:
        if token.is_stop and token.is_alpha:
            words.append(token.lemma_)
    return " ".join(words)

df["stop_words"] = df["Text"].apply(get_stop_words)
print(df["stop_words"].head())
```

```python
#MUTHURAJ_R_241801176
# EXP 3 Information Retrieval using spaCy

for i, text in enumerate(df["Text"][:3]):
    print("\nDocument", i + 1, "POS Tags:")
    doc = nlp(text)
    for token in doc[:10]:
        print(token.text, token.pos_)

vectorizer = TfidfVectorizer()
tfidf = vectorizer.fit_transform(df["clean_text"])
query = "NASA space launch"
query = preprocess(query)
query_vector = vectorizer.transform([query])

similarity = cosine_similarity(query_vector, tfidf)
similarity = similarity.flatten()
top = similarity.argsort()[-5:][::-1]
print("\nTop 5 Relevant Documents:")

for i in top:
    print("\n", df["Text"].iloc[i][:300])
```

```
0    I be some of be about the of any of about the ...
1    my be in the for a that with do anyone have on...
2    you what you about that be the will be after s...
3    it the do the not the the can do from any of t...
4    I have an which I can not with my my be that I...
Name: stop_words, dtype: object

Document 1 POS Tags:


 SPACE
I PRON
am AUX
sure ADJ
some DET
bashers NOUN
of ADP
Pens PROPN
fans NOUN
are AUX

Document 2 POS Tags:
My PRON
brother NOUN
is AUX
in ADP
the DET
market NOUN
for ADP
a DET
high ADJ
- PUNCT

Document 3 POS Tags:
```

Phil> Didn't one of the early jet fighters have these? I also think
Phil> the germans did some work on these in WWII.

The NACA came up with them before World War II. NASA is directly
descended from the NACA, with space added in.

You'll notice that I didn't mention sweep wings even though t

As for SF and advertising in space. There is a romantic episode
in Mead's "The Big Ball of Wax" where the lovers are watching
the constellation Pepsi Cola rising over the horizon and noting
the some 'stars' had slipped cause the Teamsters were on strike.

This was the inspiration for my article on

Document 3 POS Tags:

SPACE
Finally ADV
you PRON
said VERB
what PRON
you PRON
dream VERB
about ADP
. PUNCT
Mediterranean PROPN

Top 5 Relevant Documents:

I am posting this for a friend without internet access. Please inquire
to the phone number and address listed.

"Space: Teaching's Newest Frontier"
Sponsored by the Planetary Studies Foundation

The Planetary Studies Foundation is

: >Why not build a inflatable space dock.

: If you're doing large-scale satellite servicing, being able to do it in
: a pressurized hangar makes considerable sense. The question is whether
: anyone is going to be doing large-scale satellite servicing in the near
: future, to the point of justifyin

"Space Station Redesign Leader Says Cost Goal May Be
Impossible"

Today (4/6) the Washington Post ran an article with the
headline shown above. The article starts with "A leader
of the NASA team in charge of redesigning the planned
space station said yesterday the job is tough and may
be impossible.

Phil> Didn't one of the early jet fighters have these? I also think
Phil> the germans did some work on these in WWII.

The NACA came up with them before World War II. NASA is directly
descended from the NACA, with space added in.

You'll notice that I didn't mention sweep wings even though t

As for SF and advertising in space. There is a romantic episode
in Mead's "The Big Ball of Wax" where the lovers are watching
the constellation Pepsi Cola rising over the horizon and noting
the some 'stars' had slipped cause the Teamsters were on strike.

This was the inspiration for my article on

RESULT:

The information retrieval system using spaCy and TF-IDF was executed successfully. Text was preprocessed using tokenization, stop-word removal, and lemmatization; part-of-speech tags were displayed for sample documents. The query “NASA space launch” was compared with the document collection using cosine similarity, and the top five relevant documents were displayed.

EXPERIMENT 4: INFORMATION RETRIEVAL USING TF-IDF AND COSINE SIMILARITY

AIM:

To develop an information retrieval system using the 20 Newsgroups dataset by preprocessing text, removing English stop words, converting documents into TF-IDF vectors, and retrieving the most relevant documents for the query “space shuttle launch” using cosine similarity.

ALGORITHM:

1. Import the required libraries, including NLTK, Pandas, scikit-learn, and cosine similarity utilities.

2. Download the required NLTK resources and load the 20 Newsgroups training dataset, excluding headers, footers, and quoted text.

3. Store the dataset text in a Pandas DataFrame and inspect the sample records.

4. Obtain the English stop-word list and extract stop words from the document text.

5. Define a preprocessing function to convert text to lowercase, remove punctuation and non-alphanumeric characters as required, tokenize the text, and remove stop words.

6. Apply the preprocessing function to create a cleaned-text column.

7. Use TfidfVectorizer to convert the cleaned documents into TF-IDF feature vectors.

8. Enter the query “space shuttle launch” and preprocess it using the same function.

9. Transform the query into a TF-IDF vector and calculate cosine similarity with the document vectors.

10. Sort the similarity scores and display the top five most relevant documents.

11. Tokenize sample documents and display their part-of-speech tags.

```python
# EXPERIMENT-3B Muthuraj(241801176)

# Step 1
import nltk
import pandas as pd
import string
import re

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt_tab')

newsgroups_train = fetch_20newsgroups(subset='train')

df = pd.DataFrame({'text': newsgroups_train.data})

print(df.head(2).to_markdown())

newsgroups_data = fetch_20newsgroups(
    subset='train',
        remove=('headers', 'footers', 'quotes')
        )
```

...

	text
0	From: lerxst@wam.umd.edu (where's my thing) Subject: WHAT car is this!? Nntp-Posting-Host: rac3.wam.umd.edu Organization: University of Maryland, College Park Lines: 15 I was wondering if anyone out there could enlighten me on this car I saw the other day. It was a 2-door sports car, looked to be from the late 60s/ early 70s. It was called a Bricklin. The doors were really small. In addition, the front bumper was separate from the rest of the body. This is all I know. If anyone can tellme a model name, engine specs, years of production, where this car is made, history, or whatever info you have on this funky looking car, please e-mail. Thanks, - IL ---- brought to you by your neighborhood Lerxst ----
1	From: guykuo@carson.u.washington.edu (Guy Kuo) Subject: SI Clock Poll - Final Call Summary: Final call for SI clock reports Keywords: SI,acceleration,clock,upgrade Article-I.D.: shelley.1qvfo9INNc3s Organization: University of Washington Lines: 11 NNTP-Posting-Host: carson.u.washington.edu A fair number of brave souls who upgraded their SI clock oscillator have shared their experiences for this poll. Please send a brief message detailing your experiences with the procedure. Top speed attained, CPU rated speed, add on cards and adapters, heat sinks, hour of usage per day, floppy disk functionality with 800 and 1.4 m floppies are especially requested. I will be summarizing in the next two days, so please add to the network knowledge base if you have done the clock upgrade and haven't answered this poll. Thanks.

```
# EXPERIMENT-3B Muthuraj(241801176)

# Step 2
documents = newsgroups_data.data[:1000]
df = pd.DataFrame({'text': documents})

documents[0]

stop_words = set(stopwords.words('english'))

print("Sample Stop Words:")
print(list(stop_words)[:5])

def extract_stop_words(text):
    text = text.lower()
    tokens = word_tokenize(text)
    return [word for word in tokens if word in stop_words and word.isalpha()]

stop_words_list = df['text'].apply(extract_stop_words)
stop_words_list
```

```
Sample Stop Words:
['haven', 'yourselves', 'more', 'where', 'should']
```

	text
0	[i, was, if, out, there, me, on, this, i, the,...
1	[a, of, who, their, have, their, for, this, a,...
2	[my, up, the, this, after, as, a, in, i, in, t...
3	[do, you, have, i, to, some, about, this]
4	[from, by, a, my, is, that, the, are, in, the...

```
# EXPERIMENT-3B SMuthuraj(241801176)

# Step 3
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)

    tokens = word_tokenize(text)

    tokens = [w for w in tokens if w not in stopwords.words('english')]

    return ' '.join(tokens)

df['clean_text'] = df['text'].apply(preprocess)
df['clean_text']
```

	clean_text
0	wondering anyone could enlighten car saw day d...
1	fair number brave souls upgraded si clock osci...
2	well folks mac plus finally gave ghost weekend...
3	weiteks addressphone number id like get inform...
4	article cowcbnpworldstdcom tombakerworldstdcom...
...	...
995	vt vt compatible terminal external hyess modem...
996	obviously really dont know spoken cf secy stat...
997	noticed exact phenomenon occurs lciii perhaps ...

```python
# EXPERIMENT-3B Muthuraj(241801176)

# Step 4
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['clean_text'])
```

```python
# EXPERIMENT-3B Muthuraj(241801176)

# Step 5
query = "space shuttle launch"
query_clean = preprocess(query)
query_vec = vectorizer.transform([query_clean])
```

```python
# EXPERIMENT-3B Muthuraj(241801176)

# Step 6
similarities = cosine_similarity(query_vec, tfidf_matrix).flatten()

similarities[:5]
```

```
array([0.        , 0.        , 0.        , 0.        , 0.10156444])
```

```python
# EXPERIMENT-3B Muthuraj(241801176)

# Step 7
top_indices = similarities.argsort()[-5:][::-1]

print("\n🔍 Top 5 Relevant Documents for Query:", query)
for i in top_indices:
    print(f"\n📄 Doc {i+1} | Score: {similarities[i]:.4f}\n{df.loc[i, 'text'][:400]}...\n")

    for i in top_indices:
        print(df.loc[i, 'text'][:300])
```

```
    SHUTTLE LAUNCHINGS AND LANDINGS; SCHEDULES AND HOW TO SEE THEM

    Shuttle operations are discussed in the Usenet group sci.space.shuttle,
    and Ken Hollis (gandalf@pr

In fact, you probably want to avoid US Government anything for such a
project.  The pricetag is invariably too high, either in money or in
hassles.

The important thing to realize here is that the big cost of getting to
the Moon is getting into low Earth orbit.  Everything else is practically
down
Archive-name: space/new_probes
Last-modified: $Date: 93/04/01 14:39:17 $

UPCOMING PLANETARY PROBES - MISSIONS AND SCHEDULES

    Information on upcoming or currently active missions not mentioned below
    would be welcome. Sources: NASA fact sheets, Cassini Mission Design
    team, ISAS/NASDA laun
According to a Software engineering professor here, what was actually rated
level five was an ibm unit which produced part of the software for the shuttle,
by not means all of it.
```

```python
# EXPERIMENT-3B Muthuraj(241801176)

# POS Tagging
for i in range(3):
    tokens = word_tokenize(df.loc[i, 'clean_text'])
    tagged = nltk.pos_tag(tokens)
    print(f"\n📄 Document {i+1} POS Tags:")
    print(tagged[:10])
```

```
📄 Document 1 POS Tags:
[('wondering', 'VBG'), ('anyone', 'NN'), ('could', 'MD'), ('enlighten', 'VB'), ('car', 'NN'), ('saw', 'JJ'), ('day', 'NN'), ('door', 'VB'), ('sports', 'NNS'), ('car', 'NN')]

📄 Document 2 POS Tags:
[('fair', 'JJ'), ('number', 'NN'), ('brave', 'VBP'), ('souls', 'NNS'), ('upgraded', 'VBD'), ('si', 'JJ'), ('clock', 'NN'), ('oscillator', 'NN'), ('shared', 'VBD'), ('experiences', 'NNS')]

📄 Document 3 POS Tags:
[('well', 'RB'), ('folks', 'NNS'), ('mac', 'VBP'), ('plus', 'CC'), ('finally', 'RB'), ('gave', 'VBD'), ('ghost', 'VBN'), ('weekend', 'NN'), ('starting', 'VBG'), ('life', 'NN')]
```

RESULT:

The information retrieval workflow was executed using the 20 Newsgroups dataset. Text preprocessing and stop-word removal were performed, documents were represented using TF-IDF vectors, and cosine similarity was used to rank documents for the query “space shuttle launch”. The top five relevant documents and sample part-of-speech tags were displayed.

EXPERIMENT 5: INFORMATION RETRIEVAL USING spaCy, TF-IDF AND COSINE SIMILARITY

AIM:

To implement an information retrieval system using the 20 Newsgroups dataset by preprocessing text with spaCy, extracting stop words, examining part-of-speech tags, converting documents into TF-IDF vectors, and ranking documents relevant to the query “NASA space launch” using cosine similarity.

ALGORITHM:

1. Import spaCy, Pandas, fetch_20newsgroups, TfidfVectorizer, and cosine_similarity.

2. Load the English spaCy language model and fetch the 20 Newsgroups dataset, excluding headers, footers, and quoted text.

3. Select the first 500 documents and store them in a Pandas DataFrame.

4. Define a preprocessing function that converts text to lowercase, tokenizes it with spaCy, removes stop words and non-alphabetic tokens, and uses token lemmas.

5. Apply the preprocessing function to create a cleaned-text column.

6. Extract and display stop words from sample documents.

7. Display part-of-speech tags for tokens in the first three documents.

8. Use TfidfVectorizer to convert the cleaned documents into TF-IDF vectors.

9. Preprocess the query “NASA space launch” and transform it into a TF-IDF vector.

10. Calculate cosine similarity between the query vector and the document vectors.

11. Rank the similarity scores and display the top five relevant documents.

```python
# EXPERIMENT-3c Muthuraj(241801176)

# Step 1: Load Dataset

import spacy
import pandas as pd
import string

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nlp = spacy.load("en_core_web_sm")

newsgroups = fetch_20newsgroups(
    subset='all',
        remove=('headers', 'footers', 'quotes')
        )

documents = newsgroups.data[:500]
```

```python
# EXPERIMENT-3c Muthuraj(241801176)

# Step 2: Preprocessing with spaCy

def preprocess_spacy(text):
    doc = nlp(text.lower())
    tokens = [
        token.lemma_
        for token in doc
         if token.is_alpha and not token.is_stop
    ]
    return ' '.join(tokens)
```

```python
# EXPERIMENT-3C Muthuraj(241801176)

# Step 3: Apply preprocessing

df = pd.DataFrame({'Text': documents})
df['clean_text'] = df['Text'].apply(preprocess_spacy)
```

```python
# EXPERIMENT-3CMuthuraj(241801176)

# Step 4: Function to extract stop words as a space-separated string

def extract_stop_words(text):
    doc = nlp(text.lower())
    stop_words_in_text = [
        token.lemma_
        for token in doc
        if token.is_stop and token.is_alpha
    ]
    return ' '.join(stop_words_in_text)

df['stop_words'] = df['Text'].apply(extract_stop_words)
df[['stop_words']].head()
```

	stop_words
0	I be some of be about the of any of about the ...
1	my be in the for a that with do anyone have on...
2	you what you about that be the will be after s...
3	it the do the not the the can do from any of t...
4	I have an which I can not with my my be that I...

```python
# EXPERIMENT-3C Muthuraj(241801176)

# Step 5: POS Tagging example (first few docs)

for i, doc in enumerate(df['Text'][:3]):
    print(f"\nDocument {i+1} POS Tags:")
    spacy_doc = nlp(doc)
    print([(token.text, token.pos_) for token in spacy_doc[:10]])
```

```
Document 1 POS Tags:
[('\n\n', 'SPACE'), ('I', 'PRON'), ('am', 'AUX'), ('sure', 'ADJ'), ('some', 'DET'), ('bashers', 'NOUN'), ('of', 'ADP'), ('Pens', 'PROPN'), ('fans', 'NOUN'), ('are', 'AUX')]

Document 2 POS Tags:
[('My', 'PRON'), ('brother', 'NOUN'), ('is', 'AUX'), ('in', 'ADP'), ('the', 'DET'), ('market', 'NOUN'), ('for', 'ADP'), ('a', 'DET'), ('high', 'ADJ'), ('-', 'PUNCT')]

Document 3 POS Tags:
[('\n\n\n\n\t', 'SPACE'), ('Finally', 'ADV'), ('you', 'PRON'), ('said', 'VERB'), ('what', 'PRON'), ('you', 'PRON'), ('dream', 'VERB'), ('about', 'ADP'), ('.', 'PUNCT'), ('Mediterrane
```

```python
# EXPERIMENT-3C Muthuraj(241801176)
# Step 6: Vectorization using TF-IDF

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['clean_text'])
```

```
# EXPERIMENT-3C Muthuraj(241801176)
# Step 7: Query

query = "NASA space launch"
query_clean = preprocess_spacy(query)
query_vec = vectorizer.transform([query_clean])
```

```
# EXPERIMENT-3C Muthuraj(241801176)

# Step 8: Similarity & Ranking

similarities = cosine_similarity(
    query_vec,
    tfidf_matrix
).flatten()

top_indices = similarities.argsort()[-5:][::-1]
```

```
# EXPERIMENT-3C Muthuraj(241801176)

# Step 9: Display top 5 results (just the text)

print("\n🔍 Top 5 Relevant Documents:")

for i in top_indices:
    print(df.loc[i, 'Text'][:300])
    print()
```

```
🔍 Top 5 Relevant Documents:
I am posting this for a friend without internet access. Please inquire
to the phone number and address listed.
--------------------------------------------------------------------

"Space: Teaching's Newest Frontier"
Sponsored by the Planetary Studies Foundation

The Planetary Studies Foundation is

: >Why not build a inflatable space dock.

: If you're doing large-scale satellite servicing, being able to do it in
: a pressurized hangar makes considerable sense.  The question is whether
: anyone is going to be doing large-scale satellite servicing in the near
: future, to the point of justifyin

"Space Station Redesign Leader Says Cost Goal May Be
Impossible"

Today (4/6) the Washington Post ran an article with the
```

RESULT:

The information retrieval program processed the 20 Newsgroups text collection using spaCy, extracted stop words, and displayed part-of-speech tags for sample documents. The documents were converted into TF-IDF vectors, and cosine similarity was used to rank documents for the query “NASA space launch”. The top five relevant documents were displayed.

EXPERIMENT 6: ADVERTISING SALES PREDICTION AND K-MEANS CLUSTERING

AIM:

To analyze the Advertising Budget and Sales dataset by exploring the data, splitting it into training and testing sets, building and comparing Linear Regression, Polynomial Regression, and Support Vector Regression models, and applying K-Means clustering to identify groups in the advertising data.

ALGORITHM:

1. Import the required libraries, including Pandas, NumPy, Seaborn, Matplotlib, train_test_split, LinearRegression, PolynomialFeatures, make_pipeline, SVR, KMeans, and error metrics.
2. Load the Advertising Budget and Sales CSV dataset and display the first five rows.
3. Check missing values and inspect the dataset data types.
4. Select TV, Radio, and Newspaper advertising budgets as input features (X) and Sales as the target (y).
5. Visualize the relationships and distributions of the advertising budget features using a pair plot.
6. Split the data into training and testing sets using test_size=0.2 and random_state=42.
7. Train a Linear Regression model and generate predictions for the test data.
8. Train a Polynomial Regression model using PolynomialFeatures with degree 2 and generate predictions.
9. Train a Support Vector Regression model using an RBF kernel and generate predictions.
10. Evaluate the models using Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).

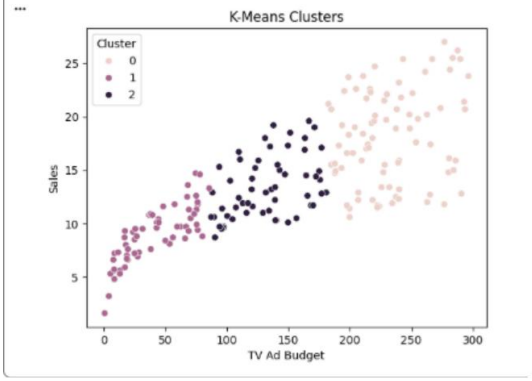
11. Compare the evaluation metrics for Linear, Polynomial, and SVR models using a bar chart.
12. Apply K-Means clustering with three clusters to the advertising dataset and add the cluster labels to the DataFrame.
13. Visualize the clusters using a scatter plot of TV advertising budget against Sales.

```
# EXERCISE-3 MUTHURAJ R (241801176)
# 9. Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X)
print(df[["TV Ad Budget ($)", "Radio Ad Budget ($)", "Newspaper Ad Budget ($)", "Sales ($)", "Cluster"]].head())
```

```
   TV Ad Budget ($)  Radio Ad Budget ($)  Newspaper Ad Budget ($)  Sales ($)  \
0             230.1                 37.8                     69.2       22.1
1              44.5                 39.3                     45.1       10.4
2              17.2                 45.9                     69.3        9.3
3             151.5                 41.3                     58.5       18.5
4             180.8                 10.8                     58.4       12.9

   Cluster
0        0
1        1
2        1
3        2
4        2
```

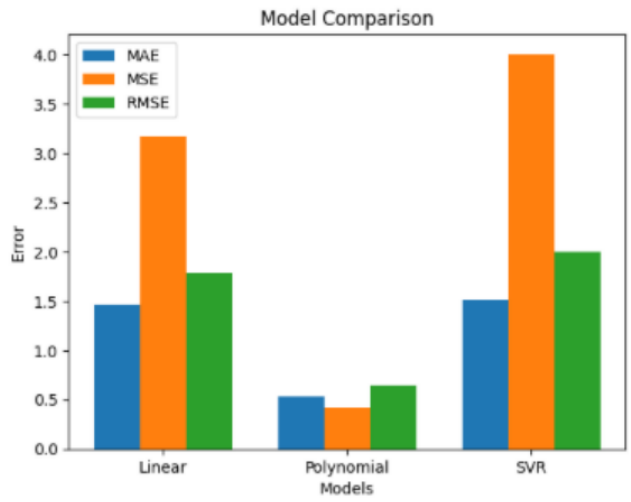
```
# EXERCISE-3 MUTHURAJ R (241801176)
# 10. Visualize Clusters
sns.scatterplot(
    x=df["TV Ad Budget ($)"],
    y=df["Sales ($)"],
    hue=df["Cluster"]
)
plt.title("K-Means Clusters")
plt.xlabel("TV Ad Budget")
plt.ylabel("Sales")
plt.show()
```



```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 5. Train Polynomial Regression
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_model.fit(X_train, y_train)
poly_pred = poly_model.predict(X_test)
print("Polynomial Regression Predictions")
print(poly_pred)
```

```
Polynomial Regression Predictions
[17.25443578 22.7193321  20.42799436  7.542709   24.37103037 12.55927164
 22.80784986  8.34371358 12.0586023  15.69024565  7.89166367  8.27082715
 11.86055971  6.16591094 10.57664393 12.3628954   6.74390963 16.65082111
 10.68661722 19.03888161 20.15223945 13.1137814   9.56273868 22.10675225
  8.96725241  7.7794437  22.40745151 12.72065973 10.25529986  6.22368636
 11.64677688 10.22431946 23.39763949  9.17403232 15.36143449 21.05675814
 10.9795286  20.23370753 11.85153256  6.58779915]
```

```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 8. Model Comparison
models = ["Linear", "Polynomial", "SVR"]
mae = [linear_mae, poly_mae, svr_mae]
mse = [linear_mse, poly_mse, svr_mse]
rmse = [linear_rmse, poly_rmse, svr_rmse]
x = np.arange(len(models))
width = 0.25
plt.bar(x - width, mae, width, label="MAE")
plt.bar(x, mse, width, label="MSE")
plt.bar(x + width, rmse, width, label="RMSE")
plt.xticks(x, models)
plt.xlabel("Models")
plt.ylabel("Error")
plt.title("Model Comparison")
plt.legend()
plt.show()
```



```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 7. Evaluate Models
linear_mae = mean_absolute_error(y_test, linear_pred)
linear_mse = mean_squared_error(y_test, linear_pred)
linear_rmse = np.sqrt(linear_mse)
poly_mae = mean_absolute_error(y_test, poly_pred)
poly_mse = mean_squared_error(y_test, poly_pred)
poly_rmse = np.sqrt(poly_mse)
svr_mae = mean_absolute_error(y_test, svr_pred)
svr_mse = mean_squared_error(y_test, svr_pred)
svr_rmse = np.sqrt(svr_mse)
print("Linear Regression")
print("MAE:", linear_mae)
print("MSE:", linear_mse)
print("RMSE:", linear_rmse)
print("\nPolynomial Regression")
print("MAE:", poly_mae)
print("MSE:", poly_mse)
print("RMSE:", poly_rmse)
print("\nSupport Vector Regression")
print("MAE:", svr_mae)
print("MSE:", svr_mse)
print("RMSE:", svr_rmse)
```

```
Linear Regression
MAE: 1.4607567168117603
MSE: 3.1740973539761033
RMSE: 1.78159966153345

Polynomial Regression
MAE: 0.5261794444043838
MSE: 0.41291022853790765
RMSE: 0.6425809120553673

Support Vector Regression
MAE: 1.514357586594181
MSE: 4.00604540443217
RMSE: 2.0015107804936174
```

```python
# EXERCISE-3 MUTHURAJ R (241801176)
# 6. Train Support Vector Regression
svr_model = SVR(kernel="rbf")
svr_model.fit(X_train, y_train)
svr_pred = svr_model.predict(X_test)
print("SVR Predictions")
print(svr_pred)
```

```
SVR Predictions
[16.82433682 19.737857   18.51883217  8.49757937 19.14449648 12.35892516
 19.35370785  8.670194   14.26079367 15.69961405  8.67459241  8.63883765
 16.10338892  7.86941414 11.80945227 13.73449615  7.82912158 16.07178231
 10.8096626  17.39870345 19.02145965 12.85124102  9.98938778 19.67786747
  9.38640699  8.22363571 19.35662438 12.9459954  10.6084116   7.91787133
 13.69013745 10.3362837  20.05661258  9.88723936 17.12640815 17.96277376
 11.10345511 18.94964504 12.02692034  7.21381614]
```

```python
# EXERCISE-3
# MUTHURAJ R-(241801176)
# 1. Import Libraries and Load Advertising Dataset
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, mean_squared_error

df = pd.read_csv("Advertising Budget and Sales.csv")
print("Dataset Loaded Successfully")
print(df.head())
```

```
Dataset Loaded Successfully
   Unnamed: 0  TV Ad Budget ($)  Radio Ad Budget ($)  Newspaper Ad Budget ($)  \
0           1             230.1                 37.8                     69.2
1           2              44.5                 39.3                     45.1
2           3              17.2                 45.9                     69.3
3           4             151.5                 41.3                     58.5
4           5             180.8                 10.8                     58.4

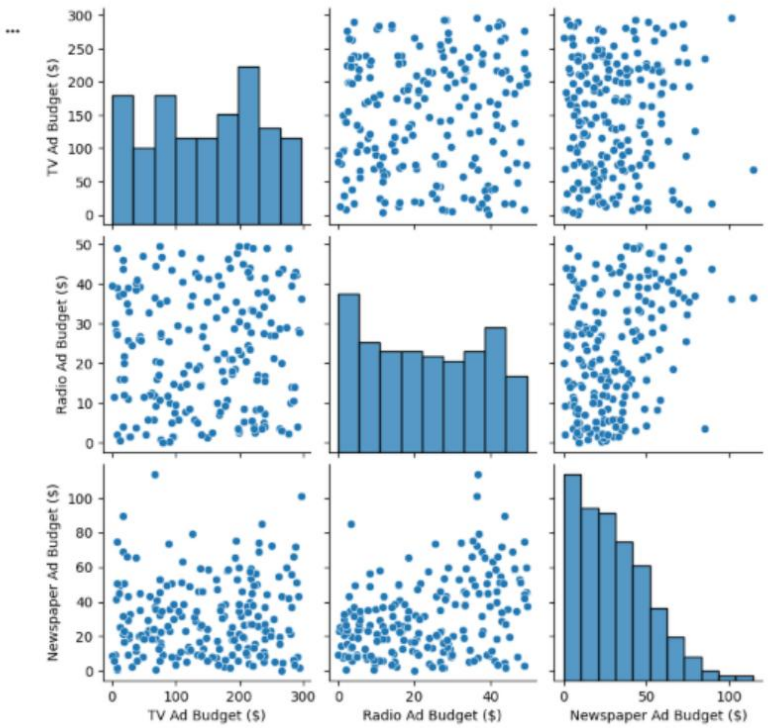
   Sales ($)
0       22.1
1       10.4
2        9.3
3       18.5
4       12.9
```

```python
# EXERCISE-3
# MUTHURAJ R (241801176)
# 4. Train Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
linear_pred = linear_model.predict(X_test)
print("Linear Regression Predictions")
print(linear_pred)
```

```
Linear Regression Predictions
[16.4080242  20.88988209 21.55384318 10.60850256 22.11237326 13.10559172
 21.05719192  7.46101034 13.60634581 15.15506967  9.04831992  6.65328312
 14.34554487  8.90349333  9.68959028 12.16494386  8.73628397 16.26507258
 10.27759582 18.83109103 19.56036653 13.25103464 12.33620695 21.30695132
  7.82740305  5.80957448 20.75753231 11.98138077  9.18349576  8.5066991
 12.46646769 10.00337695 21.3876709  12.24966368 18.26661538 20.13766267
 14.05514005 20.85411186 11.0174441   4.56899622]
```

```python
# EXERCISE-3
# MUTHURAJ R (241801176)
# 3. Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (160, 3)
Testing Data: (40, 3)
```




```
# EXERCISE-3
# MUTHURAJ R (241801176)
# 2. Explore and Preprocess Data
print("Null Values")
print(df.isnull().sum())
print("\nData Types")
print(df.dtypes)
X = df[["TV Ad Budget ($)", "Radio Ad Budget ($)", "Newspaper Ad Budget ($)"]]
y = df["Sales ($)"]
sns.pairplot(X)
plt.show()
```

```
Null Values
Unnamed: 0                0
TV Ad Budget ($)          0
Radio Ad Budget ($)       0
Newspaper Ad Budget ($)   0
Sales ($)                 0
dtype: int64

Data Types
Unnamed: 0                  int64
TV Ad Budget ($)          float64
Radio Ad Budget ($)       float64
Newspaper Ad Budget ($)   float64
Sales ($)                 float64
dtype: object
```

RESULT:

The Advertising Budget and Sales dataset was loaded and explored, with no missing values reported in the displayed output. Linear Regression, degree-2 Polynomial Regression, and Support Vector Regression were trained and their predictions generated. The models were evaluated and compared using MAE, MSE, and RMSE. K-Means clustering with three clusters was also applied, and the resulting groups were visualized by plotting TV advertising budget against Sales.

EXPERIMENT 7: HOUSE PRICE PREDICTION AND CLUSTERING

AIM:

To analyze the housing dataset, handle missing values, explore relationships among housing features, build and evaluate Linear Regression, Polynomial Regression, and Support Vector Regression models to predict median house value, and apply K-Means clustering to group housing records.

ALGORITHM:

1. Import the required libraries, including Pandas, NumPy, Seaborn, Matplotlib, train_test_split, LinearRegression, PolynomialFeatures, make_pipeline, SVR, evaluation metrics, and KMeans.

2. Load the housing CSV dataset into a Pandas DataFrame and display the first five records.

3. Inspect the dataset information, statistical description, data types, and missing values.

4. Select housing_median_age, total_rooms, total_bedrooms, and median_income as input features (X), and median_house_value as the target (y).

5. Visualize the selected features and their relationships using a pair plot.

6. Fill missing values in the selected input features using forward fill.

7. Split the dataset into training and testing sets using train_test_split with test_size=0.2 and random_state=42.

8. Train a Linear Regression model and generate predictions for the test data.

9. Train a Polynomial Regression model using PolynomialFeatures with degree 2 and generate predictions.

10. Train a Support Vector Regression model using an RBF kernel and generate predictions.

11. Evaluate each model using Mean Absolute Error (MAE) and Mean Squared Error (MSE), as shown in the output.

12. Apply K-Means clustering with three clusters to the housing data and add the cluster labels to the DataFrame.

13. Visualize the house-price clusters using a scatter plot of total_rooms against median_house_value.

```python
# EXERCISE-5 MUTHURAJ (241801176)
# Step 1: Import Required Libraries

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```python
# EXERCISE-5 MUTHURAJ (241801176)
# Step 2: Load Dataset

df = pd.read_csv(r"C:\\Users\\ADMIN\\Downloads\\archive (3)\\housing.csv")

print("First 5 rows:")
print(df.head())
```

```
First 5 rows:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                41.0        880.0           129.0
1    -122.22     37.86                21.0       7099.0          1106.0
2    -122.24     37.85                52.0       1467.0           190.0
3    -122.25     37.85                52.0       1274.0           235.0
4    -122.25     37.85                52.0       1627.0           280.0

   population  households  median_income  median_house_value ocean_proximity
0       322.0       126.0         8.3252            452600.0        NEAR BAY
1      2401.0      1138.0         8.3014            358500.0        NEAR BAY
2       496.0       177.0         7.2574            352100.0        NEAR BAY
3       558.0       219.0         5.6431            341300.0        NEAR BAY
4       565.0       259.0         3.8462            342200.0        NEAR BAY
```

```python
# EXERCISE-5 MUTHURAJ (241801176)
# Step 3: Explore Dataset

print("\nDataset Information:")
print(df.info())

print("\nStatistical Description:")
print(df.describe())
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.7 MB
None

Statistical Description:
          longitude      latitude  housing_median_age   total_rooms  \
count  20640.000000  20640.000000        20640.000000  20640.000000
mean    -119.569704     35.631861           28.639486   2635.763081
std        2.003532      2.135952           12.585558   2181.615252
min     -124.350000     32.540000            1.000000      2.000000
25%     -121.800000     33.930000           18.000000   1447.750000
50%     -118.490000     34.260000           29.000000   2127.000000
75%     -118.010000     37.710000           37.000000   3148.000000
```

```python
# EXERCISE-5 MUTHURAJ (241801176)
# Step 4: Check Missing Values

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms      207
population            0
households            0
median_income         0
median_house_value    0
ocean_proximity       0
dtype: int64
```

```python
# EXERCISE-5 MUTHURAJ (241801176)
# Step 5: Select Features and Target

X = df[['housing_median_age', 'total_rooms', 'total_bedrooms', 'median_income']]
y = df['median_house_value']

print("\nFeatures:")
print(X.head())

print("\nTarget:")
print(y.head())
```

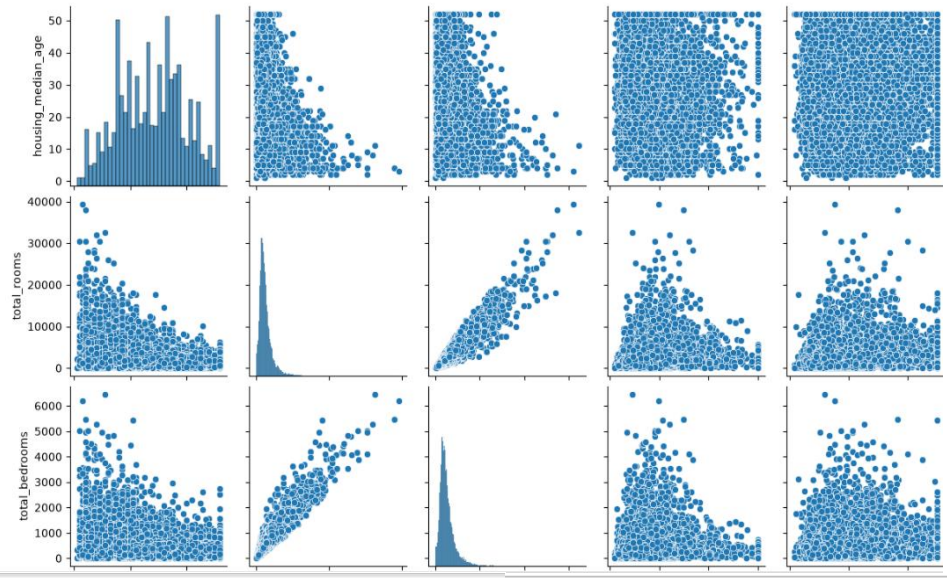
```
Features:
   housing_median_age  total_rooms  total_bedrooms  median_income
0                41.0        880.0           129.0         8.3252
1                21.0       7099.0          1106.0         8.3014
2                52.0       1467.0           190.0         7.2574
3                52.0       1274.0           235.0         5.6431
4                52.0       1627.0           280.0         3.8462

Target:
0    452600.0
1    358500.0
2    352100.0
3    341300.0
4    342200.0
Name: median_house_value, dtype: float64
```

```
# EXERCISE-5 MUTHURAJ (241801176)
# Step 6: Visualize Relationships

sns.pairplot(df[['housing_median_age', 'total_rooms',
                 'total_bedrooms', 'median_income',
                 'median_house_value']])

plt.show()
```




```
# EXERCISE-5 MUTHURAJ (241801176)
# Step 6.5: Handle Missing Values

X = X.fillna(X.median())

print("Missing values:")
print(X.isnull().sum())
```

```
Missing values:
housing_median_age    0
total_rooms           0
total_bedrooms        0
median_income         0
dtype: int64
```

```
# EXERCISE-5 MUTHURAJ (241801176)
# Step 7: Split Dataset

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# EXERCISE-5 MUTHURAJ (241801176)
# Step 8: Linear Regression

linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)
```

```
# EXERCISE-5 MUTHURAJ (241801176)
# Step 9: Polynomial Regression

polynomial_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

polynomial_model.fit(X_train, y_train)
y_pred_poly = polynomial_model.predict(X_test)

print("Polynomial Regression completed successfully!")
```

```
Polynomial Regression completed successfully!
```

```
# EXERCISE-5 SYED MUTHURAJ (241801176)
# Step 10: Support Vector Regression

svr_model = SVR(kernel='rbf')

svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)

print("SVR completed successfully!")
```

```
SVR completed successfully!
```

```python
# EXERCISE-5MUTHURAJ (241801176)
# Step 11: Model Evaluation

models = {
    "Linear Regression": y_pred_linear,
    "Polynomial Regression": y_pred_poly,
    "SVR": y_pred_svr
}

for name, prediction in models.items():
    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    print(name)
    print("MAE:", mae)
    print("MSE:", mse)
    print()
```

```
Linear Regression
MAE: 58342.91635674605
MSE: 6356567901.161352

Polynomial Regression
MAE: 62252.22762860845
MSE: 7834004738.55757

SVR
MAE: 87333.38074091132
MSE: 13739650709.806314
```

```python
# EXERCISE-5 MUTHURAJ (241801176)
# Step 12: K-Means Clustering

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)

df["Cluster"] = kmeans.labels_

print(df[["Cluster"]].head())
```

```
   Cluster
0        1
1        0
2        1
3        1
4        1
```

```python
# EXERCISE-5 MUTHURAJ (241801176)
# Step 13: Visualize Clusters

sns.scatterplot(
    data=df,
    x="total_rooms",
    y="median_house_value",
    hue="Cluster"
)

plt.title("House Price Clusters")
plt.show()
```



RESULT:

The housing dataset was loaded and explored, and missing values were checked and handled in the selected features. Linear Regression, Polynomial Regression, and Support Vector Regression models were trained to predict median house value and evaluated using MAE and MSE. K-Means clustering with three clusters was applied, and the house-price clusters were visualized using total rooms and median house value.

EXPERIMENT 8: NETFLIX CONTENT DATA ANALYSIS AND VISUALIZATION

AIM:

To analyze the Netflix content dataset using Python and Pandas, explore movies and TV shows, examine ratings, countries, release years, content growth, durations, genres and directors, and visualize content patterns and genre relationships.

ALGORITHM:

1. Import the required Python libraries and load the Netflix dataset into a Pandas DataFrame.

2. Explore the dataset by displaying its shape, columns, data types, and summary information.

3. Compare the number of Movies and TV Shows and visualize the distribution of content ratings.

4. Identify the top countries contributing Netflix content and analyze content by release year.

5. Extract the year content was added and visualize Netflix content growth over time and by month.

6. Analyze movie durations and the number of seasons for TV shows.

7. Examine the most common genres and top directors.

8. Compare ratings for Movies and TV Shows using a cross-tabulation and bar chart.

9. Build a genre co-occurrence table and visualize correlations between genres using a heatmap.

```python
# EXERCISE-7 Muthuraj R (241801176)
# STEP 1 LOAD NETFLIX DATASET
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset from the selected file path
df = pd.read_csv("/content/netflix_titles.csv")
print("Dataset Loaded Successfully!")
display(df.head())
```

```
Dataset Loaded Successfully!
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020.0	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021.0	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021.0	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerful drug lor...
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021.0	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down amo...
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021.0	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known to train I...

```python
# STEP 2 EXPLORE THE DATASET
# Muthuraj R (241801176)
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns)
print("\nDataset Information:")
df.info()
print("\nStatistical Description:")
print(df.describe(include="all"))
print("\nMissing Values:")
print(df.isnull().sum())
```

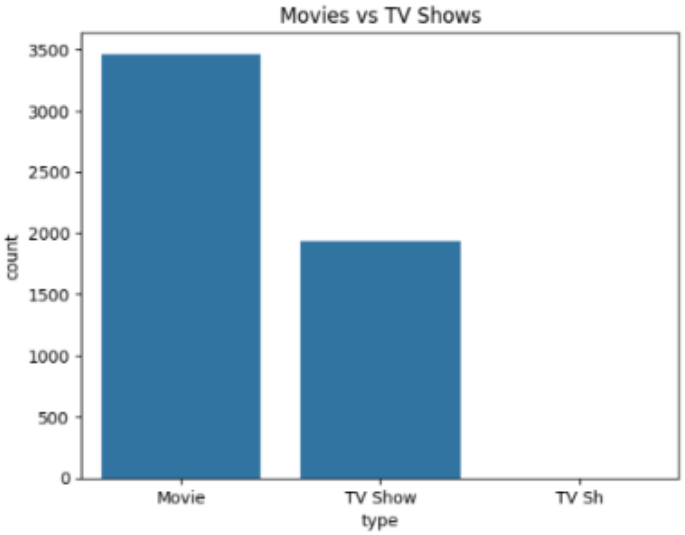
```
Shape: (5398, 12)

Columns:
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
       'release_year', 'rating', 'duration', 'listed_in', 'description'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5398 entries, 0 to 5397
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       5398 non-null   object
 1   type          5398 non-null   object
 2   title         5397 non-null   object
 3   director      3515 non-null   object
 4   cast          4903 non-null   object
 5   country       4735 non-null   object
 6   date_added    5397 non-null   object
 7   release_year  5397 non-null   float64
 8   rating        5397 non-null   object
 9   duration      5397 non-null   object
 10  listed_in     5397 non-null   object
 11  description   5397 non-null   object
dtypes: float64(1), object(11)
memory usage: 506.2+ KB
```

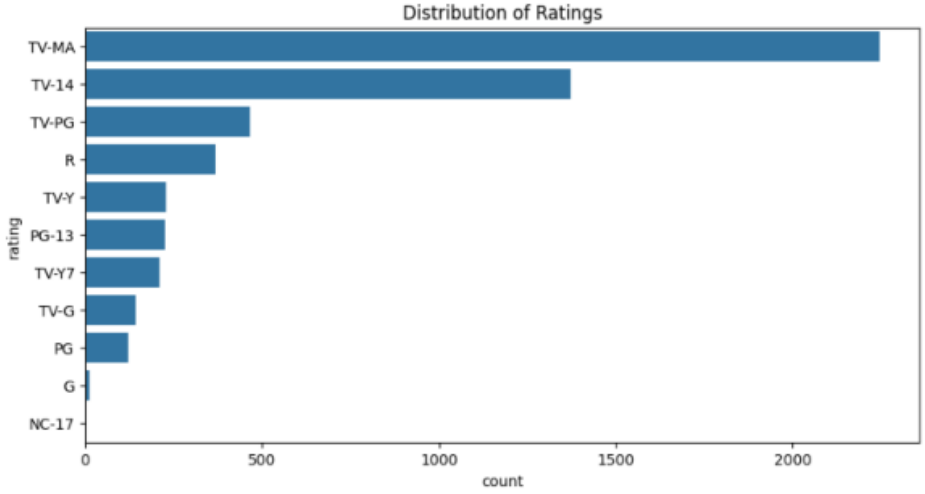
```python
# STEP 3(a) MOVIES VS TV SHOWS
# Muthuraj R (241801176)
print(df["type"].value_counts())
sns.countplot(data=df, x="type")
plt.title("Movies vs TV Shows")
plt.show()
```

```
type
Movie      3464
TV Show    1933
TV Sh         1
Name: count, dtype: int64
```



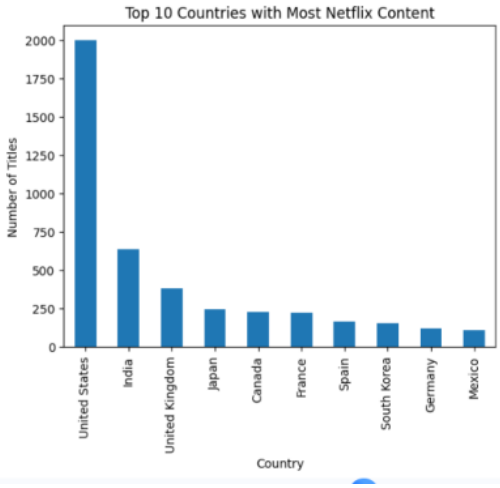

```python
# STEP 3(b) RATINGS DISTRIBUTION
# Muthuraj R (241801176)
print(df["rating"].value_counts())
plt.figure(figsize=(10, 5))
sns.countplot(data=df, y="rating", order=df["rating"].value_counts().index)
plt.title("Distribution of Ratings")
plt.show()
```

```
rating
TV-MA     2246
TV-14     1373
TV-PG      465
R          367
TV-Y       230
PG-13      226
TV-Y7      209
TV-G       144
PG         123
G           13
NC-17        1
Name: count, dtype: int64
```



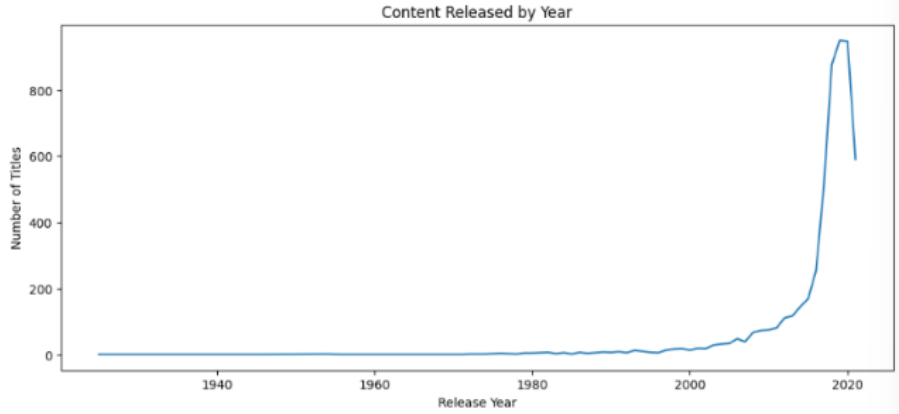
```python
# STEP 3(c) TOP COUNTRIES
# Muthuraj R (241801176)
countries = df["country"].dropna().str.split(", ").explode()
print(countries.value_counts().head(10))
countries.value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Countries with Most Netflix Content")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.show()
```

```
country
United States     1999
India              638
United Kingdom     383
Japan              244
Canada             230
France             225
Spain              166
South Korea        158
Germany            124
Mexico             111
Name: count, dtype: int64
```



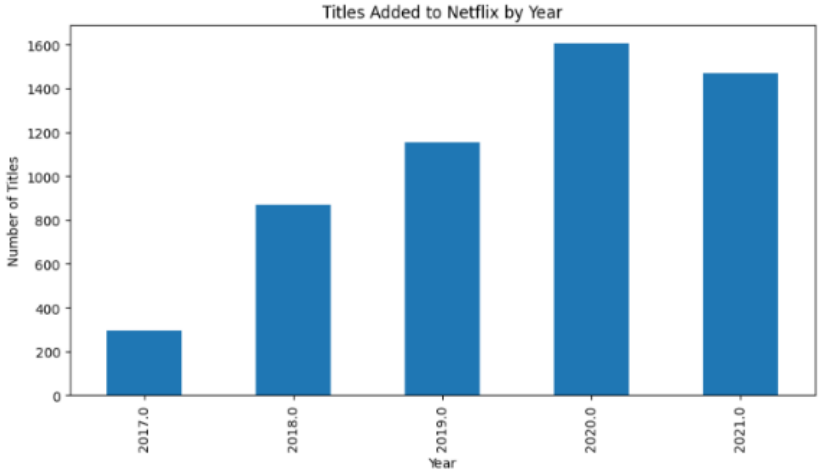

```python
# STEP 4(a) RELEASE YEAR ANALYSIS
# Muthuraj R (241801176)
print(df["release_year"].value_counts().head(10))
df["release_year"].value_counts().sort_index().plot(figsize=(12, 5))
plt.title("Content Released by Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```

```
release_year
2019.0    950
2020.0    948
2018.0    877
2021.0    591
2017.0    511
2016.0    254
2015.0    169
2014.0    145
2013.0    117
2012.0    111
Name: count, dtype: int64
```

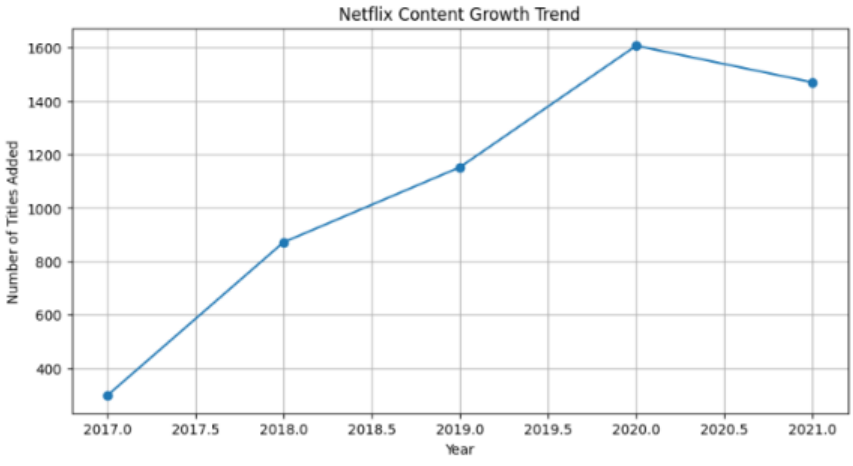


```python
# STEP 4(b) DATE ADDED YEAR
# Muthuraj R (241801176)
df["date_added"] = pd.to_datetime(df["date_added"], errors="coerce")
added_year = df["date_added"].dt.year.value_counts().sort_index()
print(added_year)
added_year.plot(kind="bar", figsize=(10, 5))
plt.title("Titles Added to Netflix by Year")
plt.xlabel("Year")
plt.ylabel("Number of Titles")
plt.show()
```

```
date_added
2017.0     297
2018.0     871
2019.0    1152
2020.0    1607
2021.0    1470
Name: count, dtype: int64
```

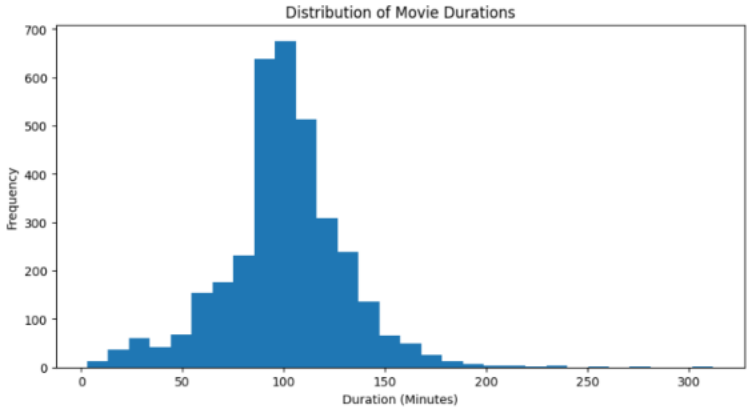



```python
# STEP 4(c) CONTENT GROWTH TREND
# Muthuraj R (241801176)
growth = df["date_added"].dt.year.value_counts().sort_index()
growth.plot(kind="line", marker="o", figsize=(10, 5))
plt.title("Netflix Content Growth Trend")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")
plt.grid()
plt.show()
```



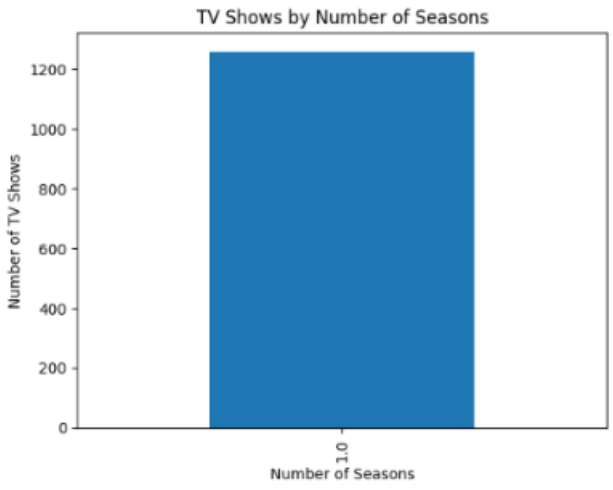
```python
# STEP 5(a) MOVIE DURATION DISTRIBUTION
movies = df[df["type"] == "Movie"].copy()
movies["duration_min"] = movies["duration"].str.replace(" min", "", regex=False).astype(float)
print(movies["duration_min"].describe())
movies["duration_min"].plot(kind="hist", bins=30, figsize=(10, 5))
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (Minutes)")
plt.show()
```

```
count    3464.000000
mean      100.387991
std        29.968087
min         3.000000
25%        87.000000
50%       100.000000
75%       116.000000
max       312.000000
Name: duration_min, dtype: float64
```



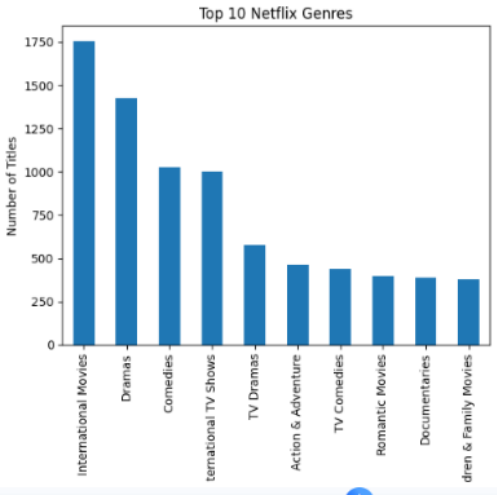

```python
# STEP 5(b) TV SHOW SEASONS
# Muthuraj R (241801176)
tv = df[df["type"] == "TV Show"].copy()
tv["seasons"] = pd.to_numeric(
    tv["duration"].str.replace(" Season", "", regex=False).str.replace(" Seasons", "", regex=False),
    errors="coerce"
)
print(tv["seasons"].value_counts().sort_index())
tv["seasons"].value_counts().sort_index().plot(kind="bar")
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```

```
seasons
1.0    1258
Name: count, dtype: int64
```



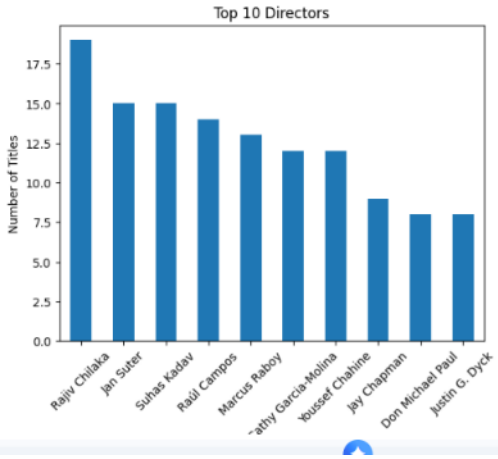
```python
# STEP 6(a) TOP GENRES
genres = df["listed_in"].dropna().str.split(", ").explode()
print(genres.value_counts().head(10))
genres.value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Netflix Genres")
plt.xlabel("Genre")
plt.ylabel("Number of Titles")
plt.show()
```

```
listed_in
International Movies        1755
Dramas                      1426
Comedies                    1025
International TV Shows      1000
TV Dramas                    576
Action & Adventure           464
TV Comedies                  436
Romantic Movies              398
Documentaries                389
Children & Family Movies     377
Name: count, dtype: int64
```



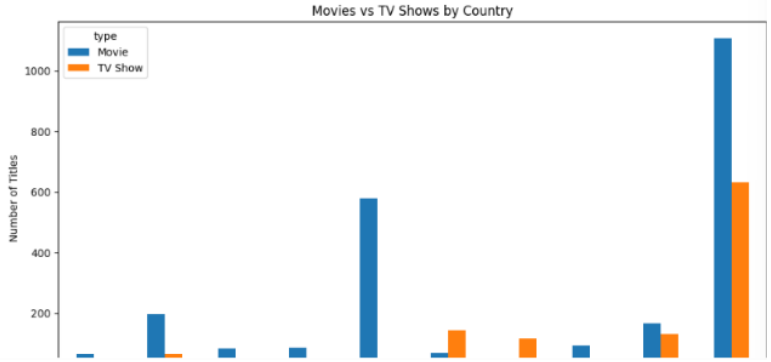

```python
# STEP 6(b) TOP DIRECTORS
# Muthuraj R (241801176)
directors = df["director"].dropna().str.split(", ").explode()
print(directors.value_counts().head(10))
directors.value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Directors")
plt.xlabel("Director")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

```
director
Rajiv Chilaka          19
Jan Suter              15
Suhas Kadav            15
Raúl Campos            14
Marcus Raboy           13
Cathy Garcia-Molina    12
Youssef Chahine        12
Jay Chapman             9
Don Michael Paul        8
Justin G. Dyck          8
Name: count, dtype: int64
```



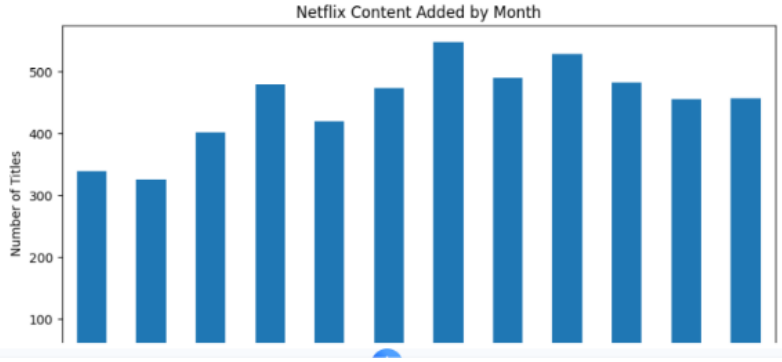
```
# Muthuraj R (241801176)
country_type = df[["country", "type"]].dropna().copy()
country_type["country"] = country_type["country"].str.split(",")
country_type = country_type.explode("country")
top_countries = country_type["country"].value_counts().head(10).index
data = country_type[country_type["country"].isin(top_countries)]
table = pd.crosstab(data["country"], data["type"])
print(table)
table.plot(kind="bar", figsize=(12, 6))
plt.title("Movies vs TV Shows by Country")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

```
type            Movie  TV Show
country
 Canada             66       30
 United States     196       65
Canada              83       51
France              86       51
India              578       46
Japan               67      144
South Korea         31      116
Spain               93       43
United Kingdom     166      132
United States     1107      632
```



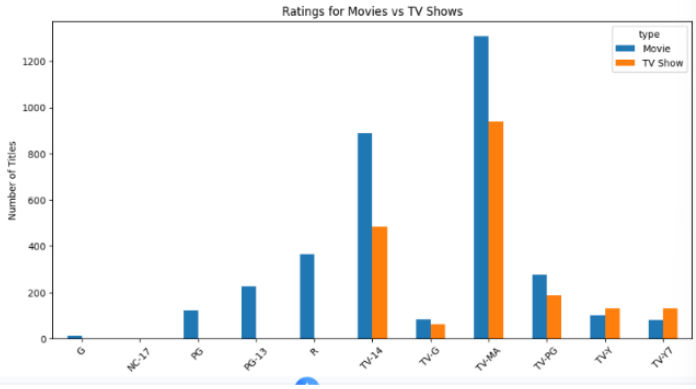

```
# STEP 8 MONTHLY CONTENT ADDITION
# Muthuraj R (241801176)
month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]
months = df["date_added"].dt.month_name()
monthly = months.value_counts().reindex(month_order)
print(monthly)
monthly.plot(kind="bar", figsize=(10, 5))
plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

```
date_added
January      338
February     325
March        402
April        479
May          420
June         473
July         548
August       490
September    528
October      482
November     455
December     457
Name: count, dtype: int64
```



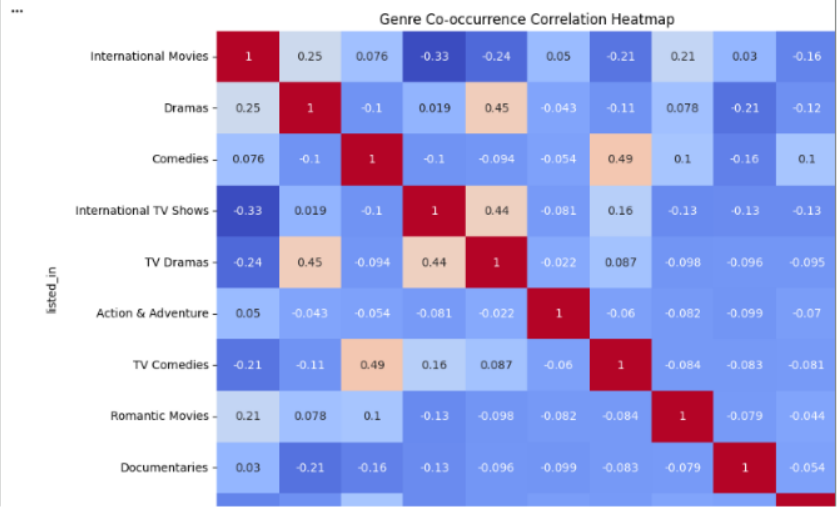
```python
# STEP 10 RATINGS BY CONTENT TYPE
# Muthuraj R (241801176)
rating_type = pd.crosstab(df["rating"], df["type"])
print(rating_type)
rating_type.plot(kind="bar", figsize=(12, 6))
plt.title("Ratings for Movies vs TV Shows")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

```
type     Movie  TV Show
rating
G           13        0
NC-17        1        0
PG         123        0
PG-13      226        0
R          366        1
TV-14      888      485
TV-G        83       61
TV-MA     1308      938
TV-PG      277      188
TV-Y       100      130
TV-Y7       79      130
```




```python
# STEP 12 GENRE CORRELATION HEATMAP
# Muthuraj R (241801176)
genres = df["listed_in"].dropna().str.split(", ").explode()
genre_list = genres.value_counts().head(10).index
genre_data = pd.DataFrame(0, index=df.index, columns=genre_list)
for i, row in df["listed_in"].fillna("").items():
    for genre in genre_list:
        if genre in row:
            genre_data.loc[i, genre] = 1

correlation = genre_data.corr()
plt.figure(figsize=(12, 8))
sns.heatmap(correlation, annot=True, cmap="coolwarm")
plt.title("Genre Co-occurrence Correlation Heatmap")
plt.show()
```



RESULT:

The Netflix dataset was explored and visualized to examine content types, ratings, countries, release years, dates added, durations, TV show seasons, genres, and directors. A comparison of ratings by content type and a genre co-occurrence correlation heatmap were generated to identify patterns in the dataset.

EXPERIMENT 9: TITANIC DATASET ANALYSIS

AIM:

To analyze the Titanic dataset using Python by inspecting dataset information and summary statistics, handling missing values, visualizing passenger-class distribution, filtering passengers based on specified conditions, and splitting the data into training and testing sets.

ALGORITHM:

1. Import the required libraries: Pandas, Seaborn, Matplotlib, SimpleImputer, and train_test_split.

2. Load the Titanic CSV dataset into a Pandas DataFrame and display the first five rows.

3. Display the dataset shape, information, and summary statistics.

4. Fill missing values in the Age column using SimpleImputer with the mean strategy.

5. Replace missing values in the Cabin column with “Unknown”.

6. Fill missing values in the Embarked column using its mode.

7. Plot the passenger class distribution using a count plot.

8. Display female passengers who survived.

9. Display third-class passengers younger than 18 years.

10. Display first-class passengers older than 40 years.

11. Display surviving passengers in the specified category.

12. Display male passengers whose fare is greater than 100.

13. Display passengers who embarked from C and belonged to passenger class 2.

14. Display passengers with SibSp greater than 2.

15. Display non-surviving passengers who had no siblings/spouses and no parents/children aboard.

16. Display the five oldest surviving passengers.

17. Display passengers who paid zero fare.

18. Split the dataset into training and testing sets with test_size=0.2 and random_state=42.

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 1. Import Libraries and Load Titanic Dataset

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

df = pd.read_csv(r"C:\\Users\\DELL\\Downloads\\archive (3)\\Titanic-Dataset.csv")

print("Dataset Loaded Successfully!")
print(df.head())
```

```
Dataset Loaded Successfully!
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 2. Understand the Data

print("Shape:", df.shape)

print("\nDataset Information")
df.info()

print("\nSummary Statistics")
print(df.describe())
```

```
Shape: (891, 12)

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

Summary Statistics
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 3. Fill Missing Age Using SimpleImputer

imputer = SimpleImputer(strategy="mean")
df[["Age"]] = imputer.fit_transform(df[["Age"]])

print(df["Age"].isnull().sum())
```

```
0
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 4. Fill Missing Cabin with "Unknown"

df["Cabin"] = df["Cabin"].fillna("Unknown")

print(df["Cabin"].isnull().sum())
```

```
0
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 5. Fill Missing Embarked with Mode

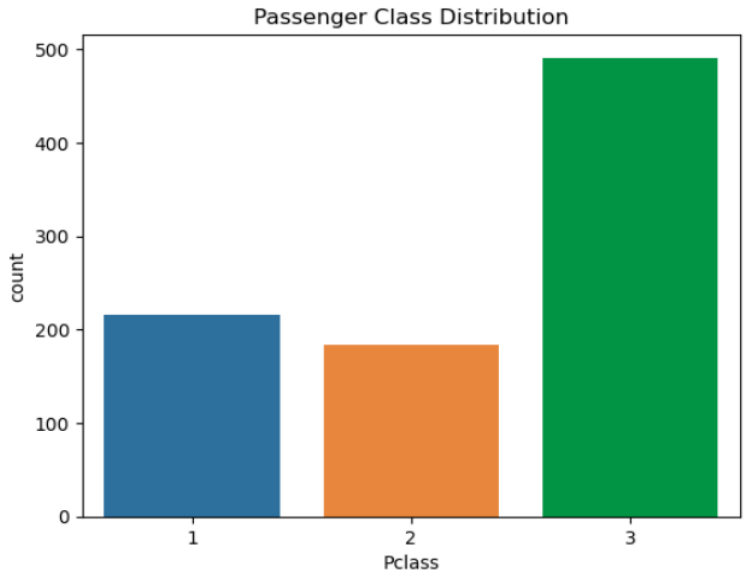
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

print(df["Embarked"].isnull().sum())
```

```
0
```

```
# EXERCISE-2 MUTHURAJ(241801176)
# 6. Passenger Class Count Plot

sns.countplot(x="Pclass", data=df)
plt.title("Passenger Class Distribution")
plt.show()
```




```
# EXERCISE-2 MUTHURAJ(241801176)
# 7. Female Passengers Who Survived

result = df[(df["Sex"] == "female") & (df["Survived"] == 1)]

print(result["Name"])
```

```
1      Cumings, Mrs. John Bradley (Florence Briggs Th...
2                                  Heikkinen, Miss. Laina
3           Futrelle, Mrs. Jacques Heath (Lily May Peel)
8      Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9                    Nasser, Mrs. Nicholas (Adele Achem)
                             ...
874                Abelson, Mrs. Samuel (Hannah Wizosky)
875                     Najib, Miss. Adele Kiamie "Jane"
879         Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880          Shelley, Mrs. William (Imanita Parrish Hall)
887                           Graham, Miss. Margaret Edith
Name: Name, Length: 233, dtype: object
```

```
# EXERCISE-2 MUTHURAJ(241801176)
# 8. 3rd Class Passengers Under Age 18

result = df[(df["Pclass"] == 3) & (df["Age"] < 18)]

print(result["Name"])
```

```
7                Palsson, Master. Gosta Leonard
10             Sandstrom, Miss. Marguerite Rut
14         Vestrom, Miss. Hulda Amanda Adolfina
16                         Rice, Master. Eugene
22                 McGowan, Miss. Anna "Annie"
                       ...
844                      Culumovic, Mr. Jeso
850    Andersson, Master. Sigvard Harald Elias
852                    Boulos, Miss. Nourelain
869             Johnson, Master. Harold Theodor
875           Najib, Miss. Adele Kiamie "Jane"
Name: Name, Length: 78, dtype: object
```

```
# EXERCISE-2 MUTHURAJ(241801176)
# 9. Class 1 Passengers Older Than 40

result = df[(df["Pclass"] == 1) & (df["Age"] > 40)]

print(result["Name"])
```

```
6                                 McCarthy, Mr. Timothy J
11                               Bonnell, Miss. Elizabeth
35                          Holverson, Mr. Alexander Oskar
52              Harper, Mrs. Henry Sleeper (Myna Haxtun)
54                          Ostby, Mr. Engelhart Cornelius
                                ...
856           Wick, Mrs. George Dennick (Mary Hitchcock)
857                                Daly, Mr. Peter Denis
862    Swift, Mrs. Frederick Joel (Margaret Welles Ba...
871     Beckwith, Mrs. Richard Leonard (Sallie Monypeny)
879        Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
Name: Name, Length: 76, dtype: object
```

```
# EXERCISE-2 MUTHURAJ(241801176)
# 10. Survived Passengers in Above Category

result = df[
    (df["Pclass"] == 1) &
    (df["Age"] > 40) &
    (df["Survived"] == 1)
]

print(result["Name"])
```

```
11                               Bonnell, Miss. Elizabeth
52              Harper, Mrs. Henry Sleeper (Myna Haxtun)
187        Romaine, Mr. Charles Hallace ("Mr C Rolmane")
194            Brown, Mrs. James Joseph (Margaret Tobin)
195                                  Lurette, Miss. Elise
268        Graham, Mrs. William Thompson (Edith Junkins)
275                     Andrews, Miss. Kornelia Theodosia
299      Baxter, Mrs. James (Helene DeLaudeniere Chaput)
337                       Burns, Miss. Elizabeth Margaret
366     Warren, Mrs. Frank Manley (Anna Sophia Atkinson)
380                                  Bidois, Miss. Rosalie
449                         Peuchen, Major. Arthur Godfrey
453                               Goldenberg, Mr. Samuel L
460                                    Anderson, Mr. Harry
496                         Eustis, Miss. Elizabeth Mussey
513       Rothschild, Mrs. Martin (Elizabeth L. Barrett)
523      Hippach, Mrs. Louis Albert (Ida Sophia Fischer)
556    Duff Gordon, Lady. (Lucille Christiana Sutherl...
571        Appleton, Mrs. Edward Dale (Charlotte Lamson)
587                      Frolicher-Stehli, Mr. Maxmillian
591      Stephenson, Mrs. Walter Bertram (Martha Eustis)
599         Duff Gordon, Sir. Cosmo Edmund ("Mr Morgan")
621                           Kimball, Mr. Edwin Nelson Jr
630                  Barkworth, Mr. Algernon Henry Wilson
645                                Harper, Mr. Henry Sleeper
647                  Simonius-Blumer, Col. Oberst Alfons
660                        Frauenthal, Dr. Henry William
707                     Calderhead, Mr. Edward Pennington
712                                Taylor, Mr. Elmer Zebley
765                  Hogeboom, Mrs. John C (Anna Andrews)
779    Robert, Mrs. Edward Scott (Elisabeth Walton Mc...
796                            Leader, Dr. Alice (Farnham)
```

```
# EXERCISE-2 MUTHURAJ(241801176)
# 11. Male Passengers Who Paid Fare Greater Than 100

result = df[(df["Sex"] == "male") & (df["Fare"] > 100)]

print(result[["Name", "Fare"]])
```

```
                                          Name      Fare
27                Fortune, Mr. Charles Alexander  263.0000
118                     Baxter, Mr. Quigg Edmond  247.5208
305               Allison, Master. Hudson Trevor  151.5500
332                    Graham, Mr. George Edward  153.4625
373                          Ringhini, Mr. Sante  135.6333
377                    Widener, Mr. Harry Elkins  211.5000
390                  Carter, Mr. William Ernest  120.0000
438                             Fortune, Mr. Mark  263.0000
505  Penasco y Castellana, Mr. Victor de Satode  108.9000
527                             Farthing, Mr. John  221.7792
544                    Douglas, Mr. Walter Donald  106.4250
550                  Thayer, Mr. John Borland Jr  110.8833
557                           Robbins, Mr. Victor  227.5250
659                    Newell, Mr. Arthur Webster  113.2750
660                 Frauenthal, Dr. Henry William  133.6500
679           Cardeza, Mr. Thomas Drake Martinez  512.3292
698                      Thayer, Mr. John Borland  110.8833
737                        Lesurer, Mr. Gustave J  512.3292
802          Carter, Master. William Thornton II  120.0000
```

```
# EXERCISE-2 MUTHURAJ(241801176)
# 12. Passengers Embarked from C and Class 2

result = df[(df["Embarked"] == "C") & (df["Pclass"] == 2)]

print(result["Name"])
```

```
9                      Nasser, Mrs. Nicholas (Adele Achem)
43               Laroche, Miss. Simonne Marie Anne Andree
122                                  Nasser, Mr. Nicholas
135                                    Richard, Mr. Emile
181                                      Pernot, Mr. Rene
292                                Levy, Mr. Rene Jacques
308                                   Abelson, Mr. Samuel
361                              del Carlo, Mr. Sebastiano
389                                   Lehmann, Miss. Bertha
473           Jerwan, Mrs. Amin S (Marie Marthe Thuillard)
547                               Padro y Manent, Mr. Julian
608    Laroche, Mrs. Joseph (Juliette Marie Louise La...
685                 Laroche, Mr. Joseph Philippe Lemercier
817                                    Mallet, Mr. Albert
827                                 Mallet, Master. Andre
866                           Duran y More, Miss. Asuncion
874                  Abelson, Mrs. Samuel (Hannah Wizosky)
Name: Name, dtype: object
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 13. Passengers with SibSp > 2

result = df[df["SibSp"] > 2]

print(result[["Name", "SibSp"]])
```

```
                                                  Name  SibSp
7                            Palsson, Master. Gosta Leonard      3
16                                    Rice, Master. Eugene       4
24                             Palsson, Miss. Torborg Danira      3
27                           Fortune, Mr. Charles Alexander       3
50                               Panula, Master. Juha Niilo       4
59                      Goodwin, Master. William Frederick        5
63                                 Skoog, Master. Harald        3
68                         Andersson, Miss. Erna Alexandra       4
71                               Goodwin, Miss. Lillian Amy      5
85   Backstrom, Mrs. Karl Alfred (Maria Mathilda Gu...         3
88                               Fortune, Miss. Mabel Helen      3
119                     Andersson, Miss. Ellis Anna Maria        4
159                            Sage, Master. Thomas Henry        8
164                          Panula, Master. Eino Viljami       4
171                                   Rice, Master. Arthur       4
176                         Lefebre, Master. Henry Forbes       3
180                          Sage, Miss. Constance Gladys       8
182                 Asplund, Master. Clarence Gustaf Hugo       4
201                                   Sage, Mr. Frederick       8
229                               Lefebre, Miss. Mathilde       3
233                         Asplund, Miss. Lillian Gertrud      4
261                       Asplund, Master. Edvin Rojj Felix     4
266                              Panula, Mr. Ernesti Arvid      4
278                                     Rice, Master. Eric      4
324                                Sage, Mr. George John Jr     8
341                           Fortune, Miss. Alice Elizabeth     3
374                              Palsson, Miss. Stina Viola      3
386                          Goodwin, Master. Sidney Leonard     5
409                                   Lefebre, Miss. Ida       3
480                           Goodwin, Master. Harold Victor     5
485                               Lefebre, Miss. Jeannie       3
541                  Andersson, Miss. Ingeborg Constanzia      4
542                     Andersson, Miss. Sigrid Elisabeth      4
634                                   Skoog, Miss. Mabel       3
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 14. Non-Survivors with No Family Aboard

result = df[
    (df["Survived"] == 0) &
    (df["SibSp"] == 0) &
    (df["Parch"] == 0)
]

print(result["Name"])
```

```
4                       Allen, Mr. William Henry
5                              Moran, Mr. James
6                        McCarthy, Mr. Timothy J
12                Saundercock, Mr. William Henry
14         Vestrom, Miss. Hulda Amanda Adolfina
                           ...
882                 Dahlberg, Miss. Gerda Ulrika
883               Banfield, Mr. Frederick James
884                      Sutehall, Mr. Henry Jr
886                       Montvila, Rev. Juozas
890                         Dooley, Mr. Patrick
Name: Name, Length: 374, dtype: object
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 15. Top 5 Oldest Passengers Who Survived

result = df[df["Survived"] == 1].nlargest(5, "Age")

print(result[["Name", "Age"]])
```

```
                                          Name   Age
630       Barkworth, Mr. Algernon Henry Wilson  80.0
275          Andrews, Miss. Kornelia Theodosia  63.0
483                       Turkula, Mrs. (Hedwig)  63.0
570                           Harris, Mr. George  62.0
829  Stone, Mrs. George Nelson (Martha Evelyn)  62.0
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 16. Passengers Who Paid Zero Fare

result = df[df["Fare"] == 0]

print(result[["Name", "Fare"]])
```

```
                                Name  Fare
179               Leonard, Mr. Lionel   0.0
263             Harrison, Mr. William   0.0
271       Tornquist, Mr. William Henry   0.0
277        Parkes, Mr. Francis "Frank"   0.0
302    Johnson, Mr. William Cahoone Jr   0.0
413     Cunningham, Mr. Alfred Fleming   0.0
466             Campbell, Mr. William   0.0
481  Frost, Mr. Anthony Wood "Archie"   0.0
597               Johnson, Mr. Alfred   0.0
633     Parr, Mr. William Henry Marsh   0.0
674        Watson, Mr. Ennis Hastings   0.0
732              Knight, Mr. Robert J   0.0
806            Andrews, Mr. Thomas Jr   0.0
815                  Fry, Mr. Richard   0.0
822   Reuchlin, Jonkheer. John George   0.0
```

```python
# EXERCISE-2 MUTHURAJ(241801176)
# 17. Split Dataset for Training and Testing

train_data, test_data = train_test_split(
    df,
    test_size=0.2,
    random_state=42
)

print("Training Data:", train_data.shape)
print("Testing Data:", test_data.shape)
```

```
Training Data: (712, 12)
Testing Data: (179, 12)
```

RESULT:

The Titanic dataset was successfully loaded and analyzed. Dataset information and summary statistics were displayed, missing values in Age, Cabin, and Embarked were handled, and passenger-class distribution was visualized. Passenger records were filtered using the specified conditions. Finally, the dataset was split into 712 training records and 179 testing records.